

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science & Engineering

ASSESSMENT TOOL 1 – CONCEPT MAPPING

Course Code	CSA0314	Course Title	Data Structures
CO Assessed	CO1	Assessment	Assessment Tool 1: Concept Mapping
Weightage	35% of CIA	Total Marks	100
CO4 Statement	<i>Apply suitable data structures and ADTs for efficient problem solving by analyzing time and space complexity. (BL2, BL3)</i>		
Student Name	Dinakaran.S	Reg. No.	1913260107
Section/Batch	Slot C / 2026	Date	06-10-2026

Group 2 **Code of Conduct**

I, Dinakaran.S, certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts	
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established	
Organization	20%	Concepts are well-structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure	
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence	
Clarity	10%	Concept map is clear, neat,	Generally clear with minor	Clarity is reduced;	Poorly presented and hard to	

		visually structured, and easy to interpret	readability issues	difficult to interpret in parts	understand	
--	--	--	-----------------------	---------------------------------------	------------	--

Big Data System Performance & Optimization

Concept Map, Performance Metrics and Optimization Strategies

Big Data Performance: Why It Matters

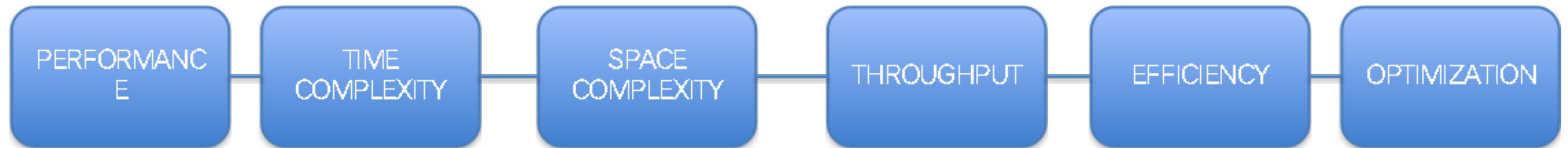
A big data system may process millions or billions of records.

Poor performance increases processing time, infrastructure cost, and user waiting time.

System design must balance speed, memory, storage, network usage, and scalability.

Key metrics: time complexity, space complexity, throughput, and efficiency.

Concept Map: Performance → Optimization



Optimization improves the metrics → improved metrics enable better system design.

Time Complexity & Space Complexity

Time Complexity: describes how processing time grows as the input size increases.

For millions of records, scalable algorithms such as $O(n \log n)$ are usually preferable to $O(n^2)$.

Space Complexity: describes how memory requirements grow with input size.

Efficient memory usage reduces RAM pressure and prevents failures when datasets become large.

Throughput & Efficiency

Throughput = amount of data or number of records processed per unit of time.

High throughput is important for batch processing, streaming, analytics, and real-time applications.

Efficiency measures how effectively CPU, memory, storage, and network resources are used.

A system with high throughput but excessive resource consumption may still be inefficient.

How Metrics Influence System Design

Low processing time → choose efficient algorithms and reduce unnecessary operations.

Low memory usage → use memory-efficient data structures and streaming techniques.

High throughput → use parallelism, partitioning, and distributed processing.

Good efficiency → balance CPU, memory, storage, and network resources.

Scalability → design components that continue to perform well as data volume increases.

Optimization Strategies

Parallel Processing – execute independent tasks simultaneously.

Data Partitioning – divide large datasets into smaller partitions for faster processing.

Caching – store frequently accessed data to avoid repeated computation or I/O.

Indexing – speed up data retrieval by reducing unnecessary searches.

Load Balancing – distribute workloads evenly across servers or processing nodes.

Compression – reduce storage and network transfer requirements.

Example: Processing 100 Million Records

Problem: A sequential system takes too long to process the entire dataset.

Approach: Partition the data and process partitions in parallel.

Use efficient algorithms and indexes to reduce unnecessary operations.

Cache frequently reused intermediate results.

Result: Lower processing time + higher throughput + better resource utilization.

Conclusion

Performance is a combination of processing speed, memory use, throughput, and resource efficiency.

Time and space complexity directly affect scalability.

Optimization strategies should target the actual performance bottleneck.

Parallel processing, partitioning, caching, indexing, and load balancing help build scalable big data systems.

The ultimate goal is high throughput with reasonable resource consumption and reliable performance.